

Class starts in 5 minutes

Agenda

↳ REST

↳ Fakestore APIs

↳ Models intro.

REST

↳ Representational State Transfer]

Suggestions:

- ① APIs should be structured around the resources that they are trying to work upon.

eg) /upload-video- X
 └ resource

/video → endpoint
└

CRUD
+++

/user]
/user /id >]
└ user

For describing the action of the API, use HTTP verbs

└ GET	└ PUT
└ POST	└ PATCH
└ DELETE	

GET user/1
DELETE user/1

GET product/1
DELETE product/1

Methods

① GET

↳ get the details of the resource
eg) GET /product/1

② POST

↳ create a resource.]

POST /product
{
 ...
}

③

{ id: 1 name: iPhone desc: iOS phone ... }	⇒	{ id: 1 name: iPhone 12 desc: iOS 15 phone ... }
---	---	--

PUT → Replace data or to create data

PUT /product/1 { id: 1
 name: iPhone 7S } → Complete replacement

① Patch

↳ partial update

id: 1

name: iPhone 15

desc: abc

→

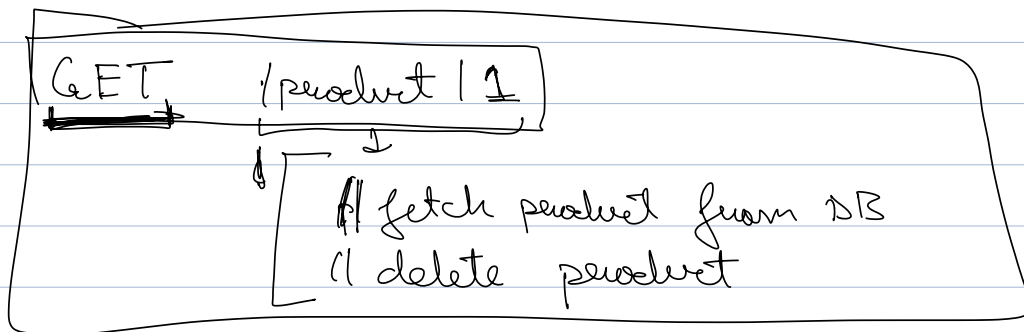
Patch /product/1

{ desc: new des }

id: 1

name: iPhone 15

desc: new ~



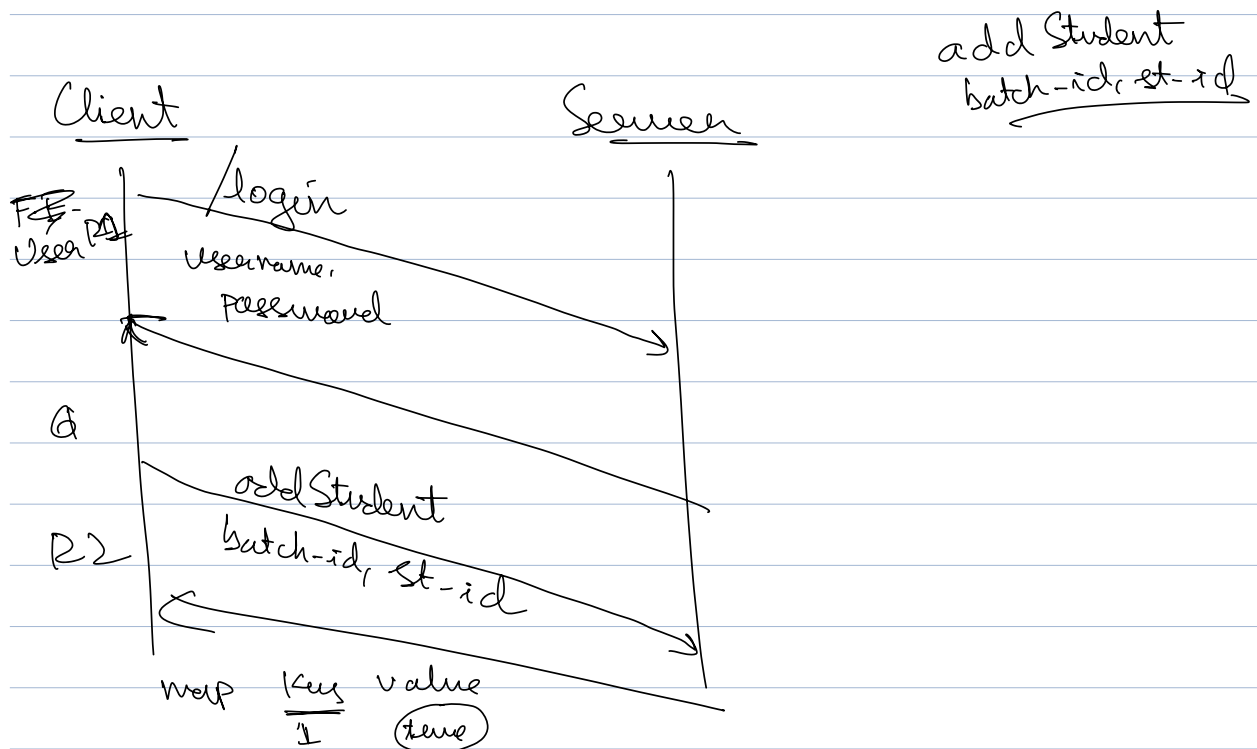
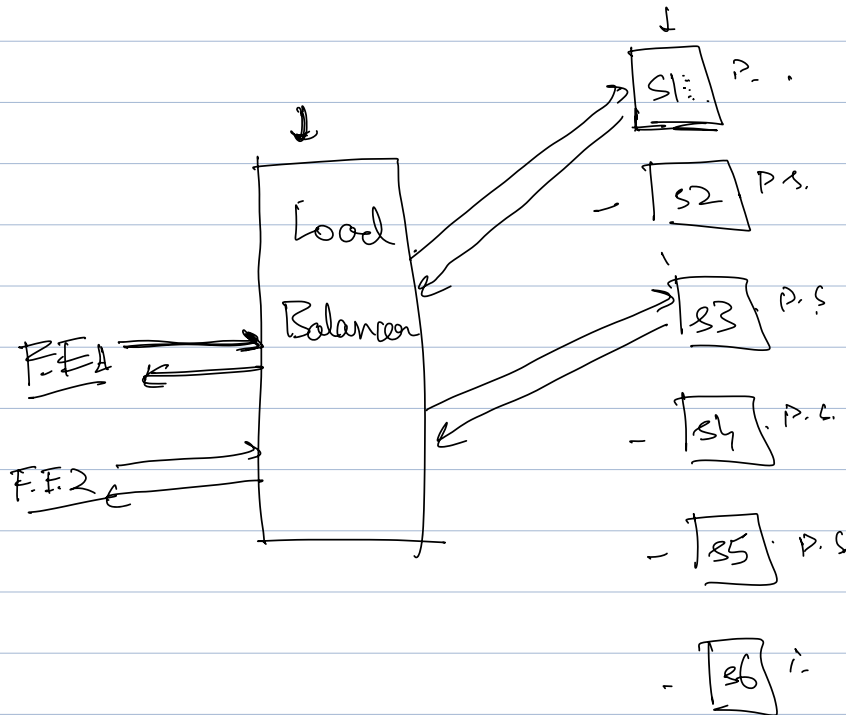
② APIs should be stateless. ←

a) Any api request should be equally serviceable via any of the servers.

b) Every api request should be self sufficient → it should contain every information that is required by the server

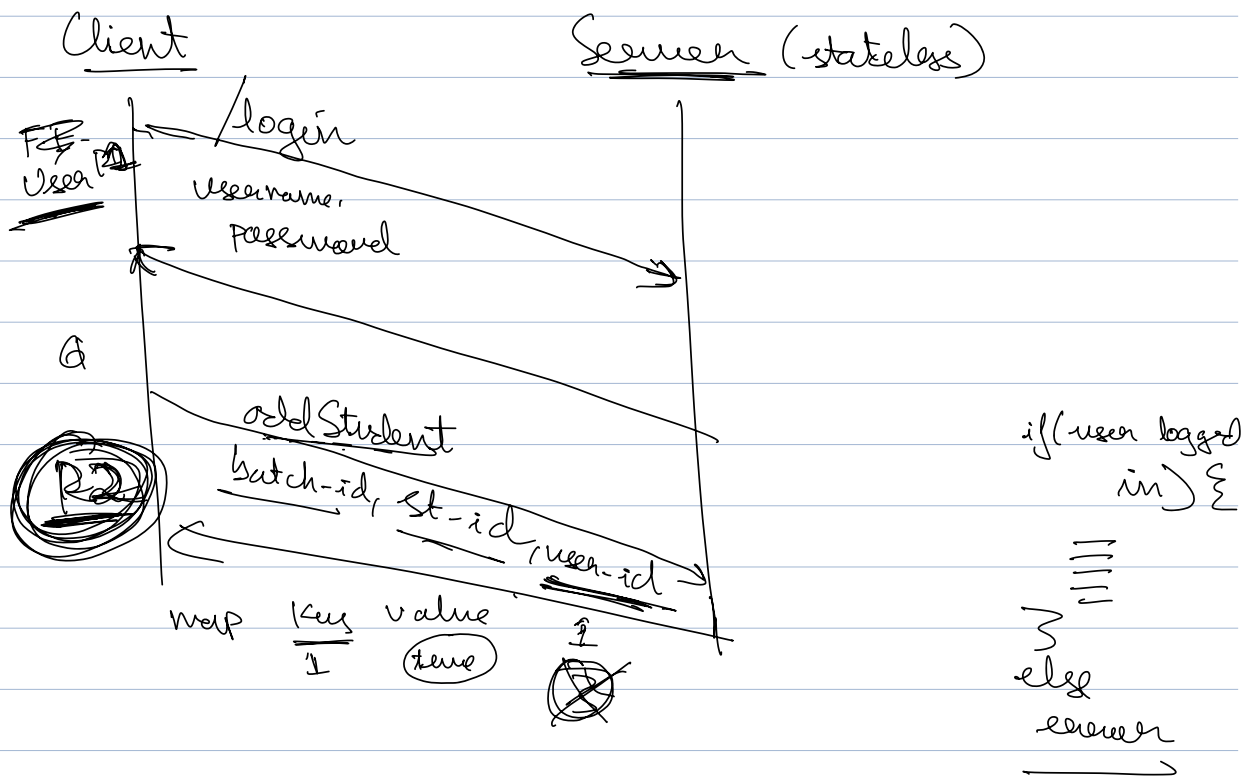
c) An api request should never assume anything or it should not assume

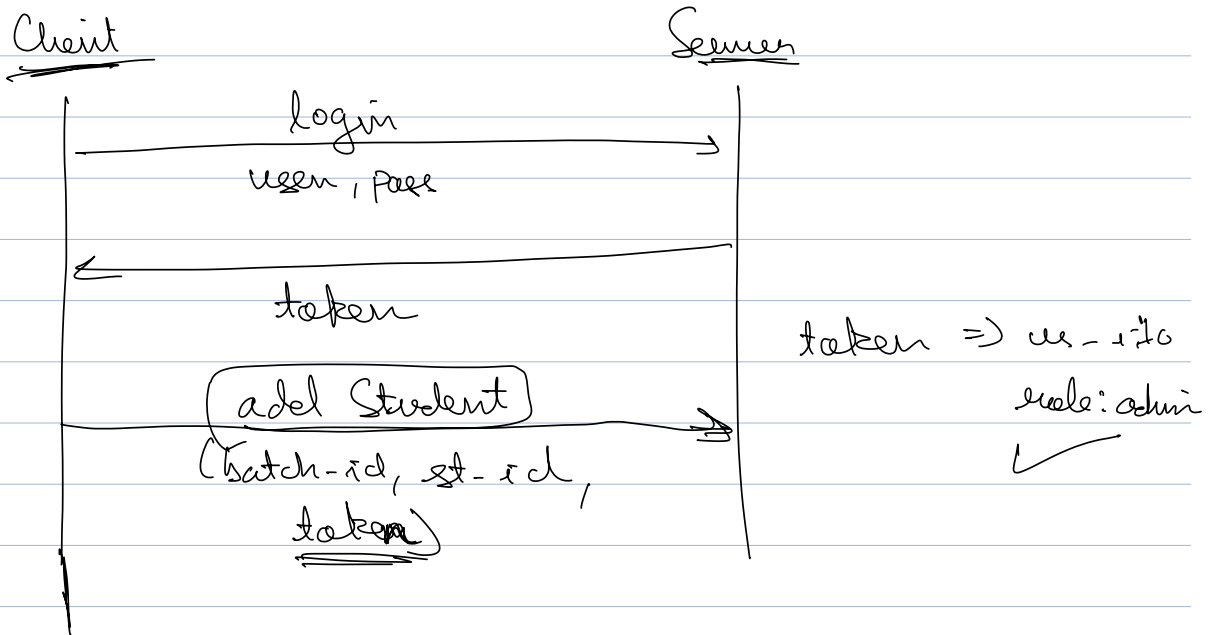
a previous request has been sent -



To make our server stateless, ensure no data is within the server. It is at a centralized place eg) a database.

Benefit => System has become scalable.

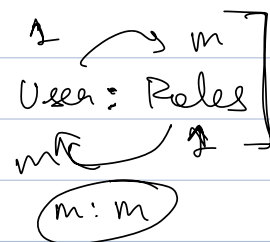




Break : 10:31 → 10:37 (6 minutes)

③ Don't have chatty APIs.

GET /user [1]
 { id: 1, name: '...', phone: '...', email: '...' }



user		role - user	
id	name	user-id	role-id
1	admin	1	7
2	TA	1	6
...		2	3
		3	4

GET /roles
 GET /user /role [1]

→ chatty APIs
 ↳ Don't do this

There is a one on one mapping between api
 resource & DB table. X

GET /user [1]

↳ { id: — ,
 name: — ,
 email: — ,
 phone: — }

Quelles: ['TA', 'Instruction']

↳ not part of user table

↳ At the expense of making my BE logic complicated,
I am reducing the no. of API calls.

- ↳ Don't have chatty APIs.

④ Rest is independent of response data types.

F.E. sending data → B.F.
 ← receiving data

JSON $\Rightarrow$ JavaScript Object Notation

$\Rightarrow$ Key-value pair / dictionary / map

(
id: 1,
name: Antik,
email: _____)

① Fasten

(2) Readable

XML

 $\langle \text{user} \rangle$ $\langle id \rangle \neq \langle 1id \rangle$
$$\langle \text{name} \rangle \sim \langle !\text{name} \rangle$$

$\langle \text{email} \rangle \rightsquigarrow \langle / \text{email} \rangle$

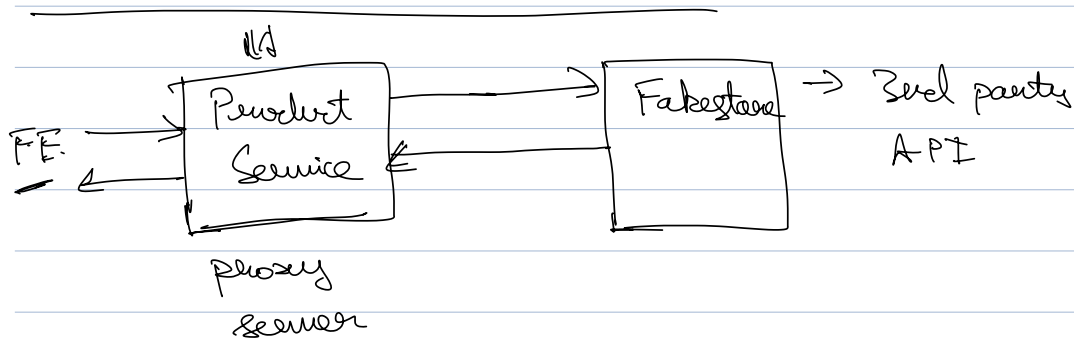
 $\angle(\mu_{\text{gen}})$

Protobufs

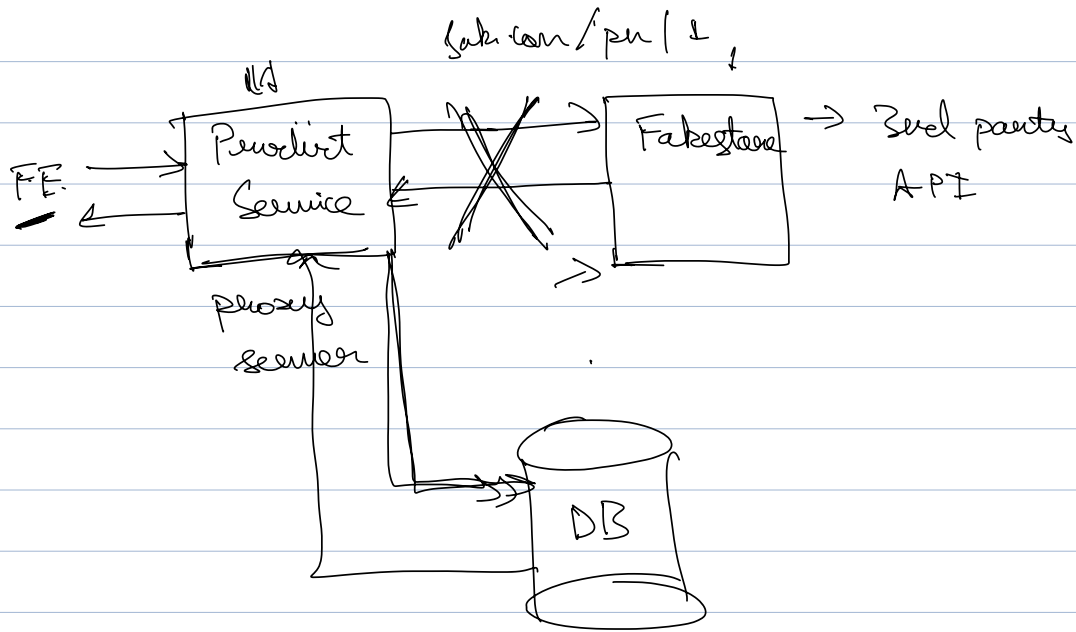
- ↳ Faster than JSON]
- ↳ But, not human readable.]

Summary

- ① API should revolve around resources
- ② Lower APIs should be self sufficient or contain all the data required to serve that request
 - ↳ stateless
- ③ Don't have chatty APIs
- ④ Independent of response data type.



why? => so that we can learn how to make api
calls from our app.



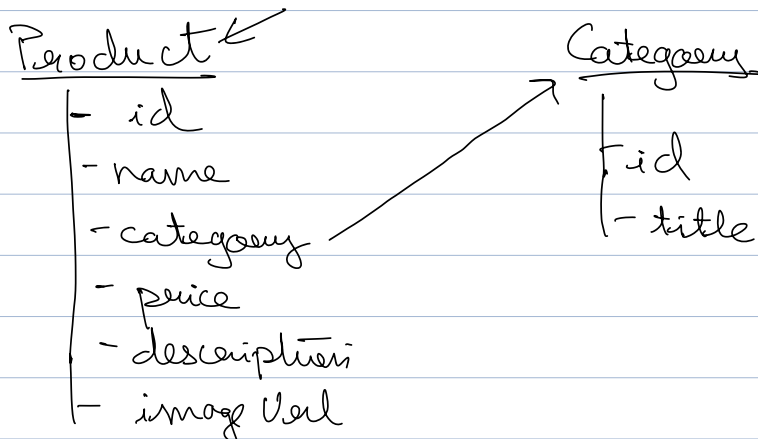
Models

MVC

↳ Model

↳ whatever you will store in DB will be represented by these models.

Products Service



Doubts

GET /user/1

→ user

id | name | email

roles

{

id: 1,

name: Sofia,

email: sofia@guail.com

}

user: roles
n: m

↓

roles - user

user-id	roles-id
1	1
1	2

↓

GET /user/~~end~~ 1 /role

1, 2

GET /roles

1 → TA
2 → Inst

3 → —

Sofia (em) TA, Inst

↓

GET /user/1 { id: 1, name: Sofia, email: sofia@guail.com, roles: [TA, Inst] }

REST Full API

@Get Mappin

getProduct By Id()